Interview Guide - explain ClinicFlow AI simply

30-second pitch

"ClinicFlow AI is a web app for GP clinics. A patient types their symptoms in plain English. AI turns that into a structured note for the doctor with an urgency level: emergency, urgent or routine. Safety rules make sure emergencies are always caught. The patient books the right type of appointment, and staff see a queue sorted by urgency. It's built with Python FastAPI, Google Gemini and SQLite, runs in Docker, and has 20 automated tests."

How it works (5 steps)

1. Patient fills the form -> `POST /api/intake`.
2. **Safety rules** scan for red flags (chest pain, stroke, suicide...).
3. **AI (Gemini)** gets a carefully designed prompt and returns JSON: urgency, summary, symptoms, appointment type, questions, patient message.
4. The code **validates** the JSON. If the AI fails -> retry -> **offline rule engine**. Red flags **override** the AI.
5. Result is saved in **SQLite**, the patient books a slot, staff see it in the **dashboard**.

Likely questions and simple answers

Why this project? The company runs GP clinics. Phone triage and booking is a daily, real pain point, and it lets me show a web app, automation and AI together.

Why FastAPI? It's simple, fast, Python (matches the Python task), validates inputs automatically with Pydantic and creates API docs at `/docs` for free.

Why plain HTML/JS instead of React? No build step, easy to deploy and explain. For this size of app React would add complexity without real benefit.

Why SQLite? Zero setup and it's built into Python. For many users I'd switch to PostgreSQL. The data layer is one file, so that's an easy change.

What if the AI gives a wrong answer? Three layers: (1) the prompt tells it to choose the more urgent level when unsure, (2) rule-based red flags always override it, (3) the output must match a strict schema, or we retry and fall back to the offline engine.

What is prompt injection and how did you handle it? A user writes instructions inside their input, like "ignore previous instructions". I wrap patient text in `<patient_input>` tags and tell the model it's data, not instructions. The safety rules also run regardless. One evaluation case tests exactly this.

How did you improve your prompts? v1 vague -> free text and diagnosis. v2 JSON + no diagnosis. v3 definitions, tie-breaker rule, few-shot example, injection defence, low temperature, JSON mode. I measured each change with `scripts/evaluate.py`.

What is "under-triage"? Saying a case is less urgent than it really is. In healthcare that's the dangerous mistake, so my evaluation treats it as a must-be-zero metric.

Is it a diagnosis tool? No. It only describes symptoms and urgency. The doctor makes all medical decisions. That keeps it safe.

How is it deployed? One Docker container. `render.yaml` deploys it on Render for free. Health check at `/api/health`. GitHub Actions runs tests, the evaluation script and a Docker build on every push.

Privacy? Demo data only. For real patients: Australian Privacy Act, Australian hosting, encryption, staff accounts, audit logs and consent.

What would you add next? SMS/email confirmations (Twilio/SendGrid), integration with clinic software (HotDoc / Best Practice), multi-language input, PostgreSQL, proper staff logins and rate limiting.

Which part was hardest? Making AI output reliable: getting consistent JSON and safe urgency levels. I solved it with prompt design, schema validation, a fallback and safety rules.

Files to know

File	One line
`app/main.py`	All API endpoints
`app/ai_engine.py`	Calls the AI, validates it, falls back, applies the safety override
`app/prompts.py`	The prompt (v3)
`app/safety.py`	Red-flag rules
`app/database.py`	SQLite tables and queries
`app/slots.py`	Appointment time slots
`static/`	Patient page and staff dashboard
`tests/`	20 automated tests
`scripts/evaluate.py`	Prompt evaluation with 12 cases